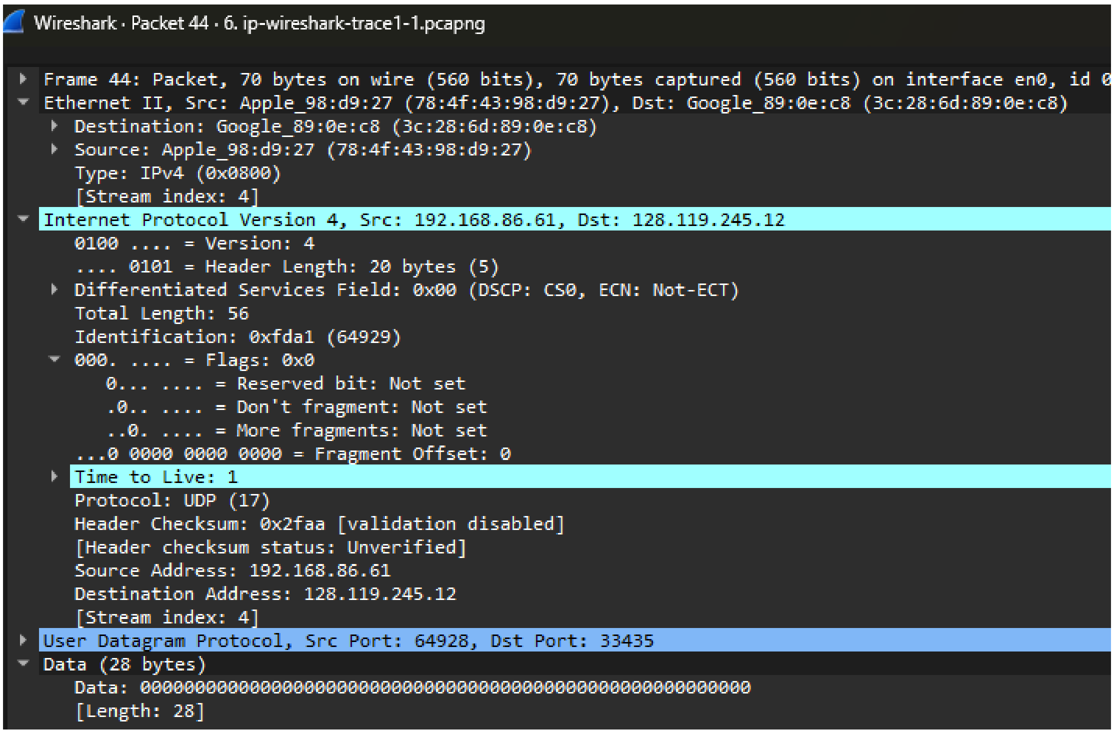
Part 1: Basic IPv4



1. Select the first UDP segment sent by your computer via the traceroute command to gaia.cs.umass.edu. (Hint: this is 44th packet in the trace file in the *ip wireshark-trace1-1.pcapng* file in footnote 2). Expand the Internet Protocol part of the packet in the packet details window. What is the IP address of your computer?

44 1.865637 192.168.86.61 128.119.245.12 UDP 70 64928 → 33435 Len=28

Source 192.168.86.61

2. What is the value in the time-to-live (TTL) field in this IPv4 datagram's header?

Time to Live: 1

3. What is the value in the upper layer protocol field in this IPv4 datagram's header? [Note: the answers for Linux/MacOS differ from Windows here].

Protocol: UDP (17)

4. How many bytes are in the IP header?

.... 0101 = Header Length: 20 bytes (5)

5. How many bytes are in the payload of the IP datagram? Explain how you determined the number of payload bytes.

Total Length: 56

.... 0101 = Header Length: 20 bytes (5)

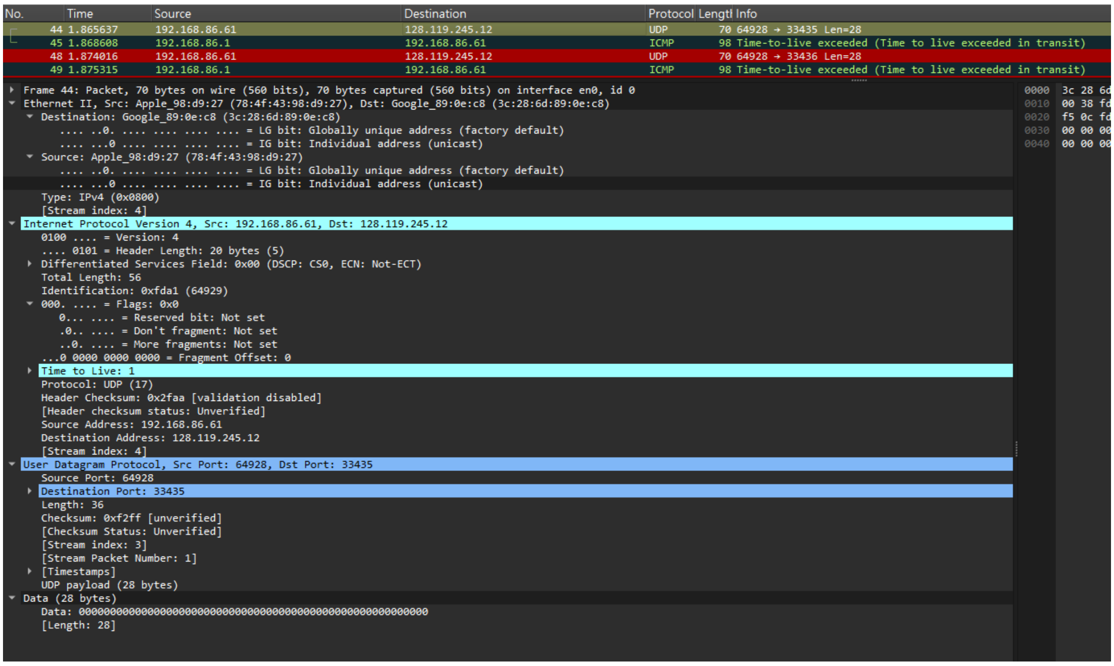
56 – 20 = 36 bytes

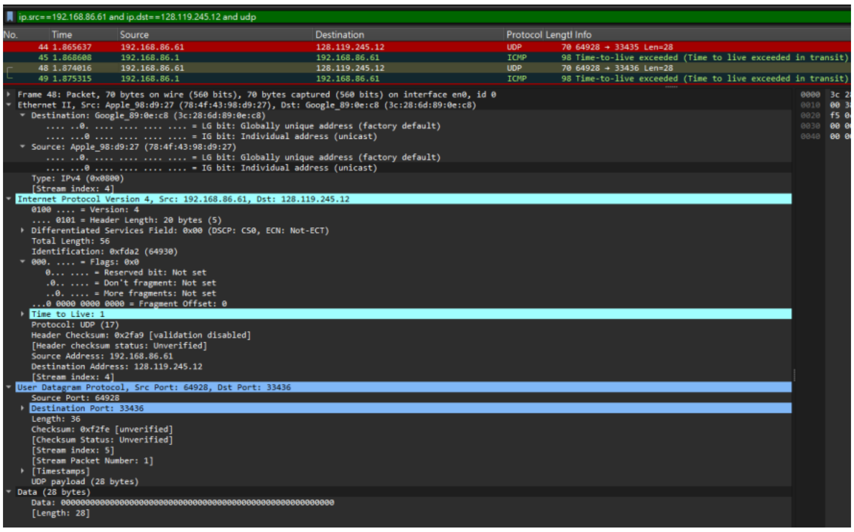
6. Has this IP datagram been fragmented? Explain how you determined whether or not the datagram has been fragmented.

NO

..0. .... = More fragments: Not set

...0 0000 0000 0000 = Fragment Offset: 0





7. Which fields in the IP datagram *always* change from one datagram to the next within this series of UDP segments sent by your computer destined to 128.119.245.12, via traceroute? Why?

Time to Live & Header Checksum & Identification

Time to Live (TTL)： 這是 traceroute (路由追蹤) 的核心運作原理！它會故意發送一連串的封包，並將 TTL 的數值依序設定為 1、2、3... 逐次遞增。這樣才能讓封包在經過第一台、第二台、第三台路由器時剛好「壽終正寢（超時）」，藉此逼迫沿途的路由器回傳錯誤訊息，進而探測出整個傳輸路徑。

Identification (識別碼)： 你的電腦作業系統每產生並發送一個新的 IP 封包，就會賦予它一個獨立且（通常是）連續遞增的編號，用來在網路上唯一辨識這個封包，所以每一筆發出去的封包編號當然都會不一樣。

Header Checksum (標頭檢查碼)： 檢查碼是根據 IP 標頭內的所有資料數值計算出來的。既然前面提到的 TTL 和 Identification 的數值都在改變，代表整個 IP 標頭的內容就不一樣了，所以檢查碼理所當然必須跟著重新計算並改變。

8. Which fields in this sequence of IP datagrams (containing UDP segments) stay constant? Why?

Version、**Header Length**、**Source Address**、**Destination Address** 這些欄位

因為都是用同一台電腦發給同一個目的地

9. Describe the pattern you see in the values in the Identification field of the IP datagrams being sent by your computer.

通常都是 +1

Identification: 0xfda1 (64929)

Identification: 0xfda2 (64930)

Identification: 0xfda3 (64931)

10. What is the upper layer protocol specified in the IP datagrams returned from the routers? [Note: the answers for Linux/MacOS differ from Windows here].

Internet Protocol Version 4, Src: 10.0.0.1, Dst: 192.168.86.61

Protocol: ICMP (1)

```
Frame 53: Packet, 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface en0, id 0
Ethernet II, Src: Google_89:0e:c8 (3c:28:6d:89:0e:c8), Dst: Apple_98:d9:27 (78:4f:43:98:d9:27)
  Destination: Apple_98:d9:27 (78:4f:43:98:d9:27)
    .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
    .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
  Source: Google_89:0e:c8 (3c:28:6d:89:0e:c8)
    .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
    .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
  Type: IPv4 (0x0800)
  [Stream index: 4]
Internet Protocol Version 4, Src: 10.0.0.1, Dst: 192.168.86.61
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  Differentiated Services Field: 0xc0 (DSCP: CS6, ECN: Not-ECT)
  Total Length: 84
  Identification: 0xd5c3 (54723)
  000. .... = Flags: 0x0
    0... .... = Reserved bit: Not set
    .0.. .... = Don't fragment: Not set
    ..0. .... = More fragments: Not set
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 63
  Protocol: ICMP (1)
  Header Checksum: 0x843f [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 10.0.0.1
  Destination Address: 192.168.86.61
  [Stream index: 6]
Internet Control Message Protocol
```

11. Are the values in the Identification fields (across the sequence of all of ICMP packets from all of the routers) similar in behavior to your answer to question 9 above?

53 Identification: 0xd5c3 (54723)

56 Identification: 0xfda5 (64933)

57 Identification: 0xd5cc (54732)

58 Identification: 0xfda6 (64934)

十六進位數字（0x...）完全是亂跳的，根本沒有連續 +1 的遞增規律

12. Are the values of the TTL fields similar, across all of ICMP packets from all of the routers?

53 Time to Live: 63

56 Time to Live: 2

57 Time to Live: 63

58 Time to Live: 2

61 Time to Live: 253

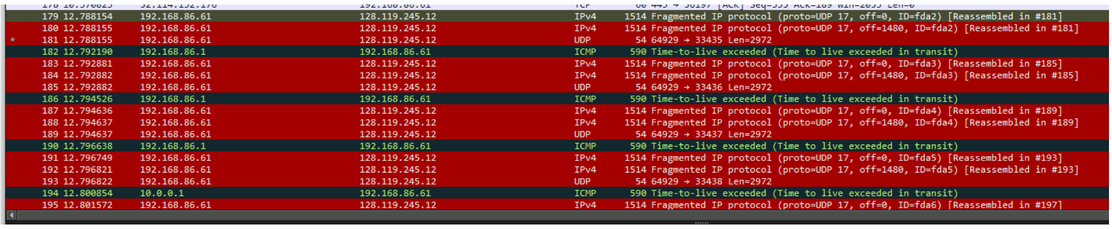
不一樣 (No)。

當沿途的路由器把 TTL 扣到 0 時，它會自己產生一個全新的 ICMP 封包寄回給你。這個回傳的 TTL 會不一樣，主要有兩個原因：

1. **作業系統預設值不同：** 不同的路由器（Cisco、Juniper 或 Linux 系統等）在產生新的 ICMP 封包時，給予的「初始 TTL」本來就不一樣（常見的有 64、128 或 255）。
2. **距離（跳數）不同：** 當這個 ICMP 封包從路由器出發，沿著網路原路返回你的電腦時，每經過一個節點，TTL 就會被扣掉 1。**越遠的路由器，回程經過的節點就越多，被扣減的次數也越多。**

Part 2: Fragmentation

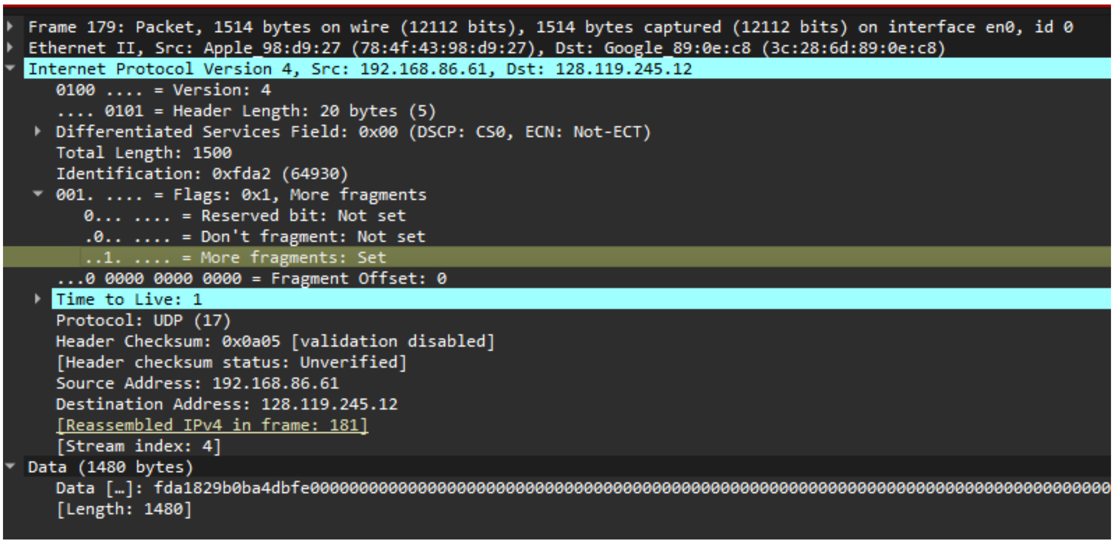
13. Find the first IP datagram containing the first part of the segment sent to 128.119.245.12 sent by your computer via the traceroute command to gaia.cs.umass.edu, *after* you specified that the traceroute packet length should be 3000. (Hint: This is packet 179 in the *ip-wireshark-trace1-1.pcapng* trace file in footnote 2. Packets 179, 180, and 181 are three IP datagrams created by fragmenting the first single 3000-byte UDP segment sent to 128.119.145.12). Has that segment been fragmented across more than one IP datagram? (Hint: the answer is yes !) [4]



YES

14. What information in the IP header indicates that this datagram been fragmented?

在實際的 IP 標頭中，Flags（旗標）欄位裡面的 More fragments (MF) 旗標會被設定為 1 (Set)。這代表該封包後面還有其他的碎塊。



15. What information in the IP header for this packet indicates whether this is the first fragment versus a latter fragment?

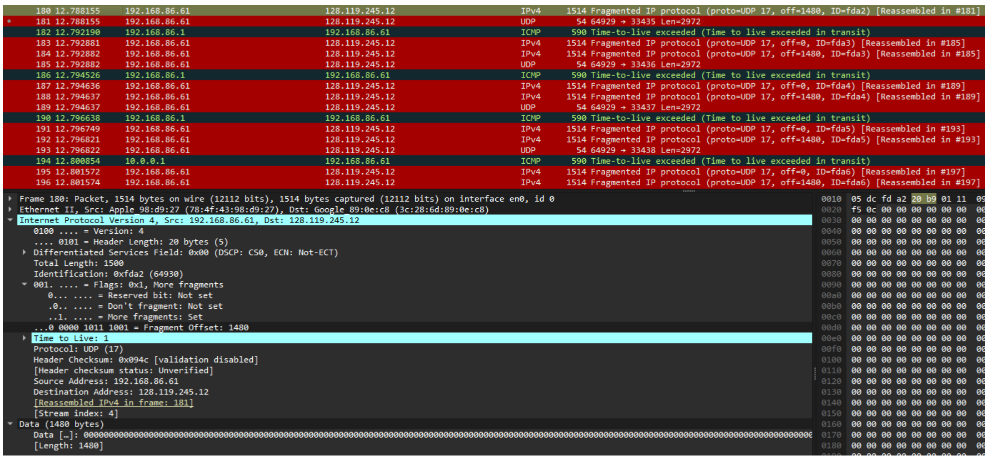
觀察第 179 號封包後方的資訊 off=0。IP 標頭中的 **Fragment Offset (分片偏移量) 數值為 0**，代表這是這組資料的最前端，也就是第一個分片。

16. How many bytes are there in is this IP datagram (header plus payload)?

最上面的 Length 欄位顯示為 1514，但這是包含了最底層 Ethernet 標頭 (14 bytes) 的大小。將 1514 減去 14，可以得知整個 IP 封包的 Total Length 為 1500 bytes。實際上資訊也寫：Total Length: 1500

17. Now inspect the datagram containing the second fragment of the fragmented UDP segment. What information in the IP header indicates that this is *not* the first datagram fragment?

它的 **Fragment Offset 數值大於 0**（從紀錄中可以看到 off=1480）。只要偏移量不是 0，就代表它絕對不是第一個分片。



18. What fields change in the IP header between the first and second fragment?

主要改變的欄位有兩個：

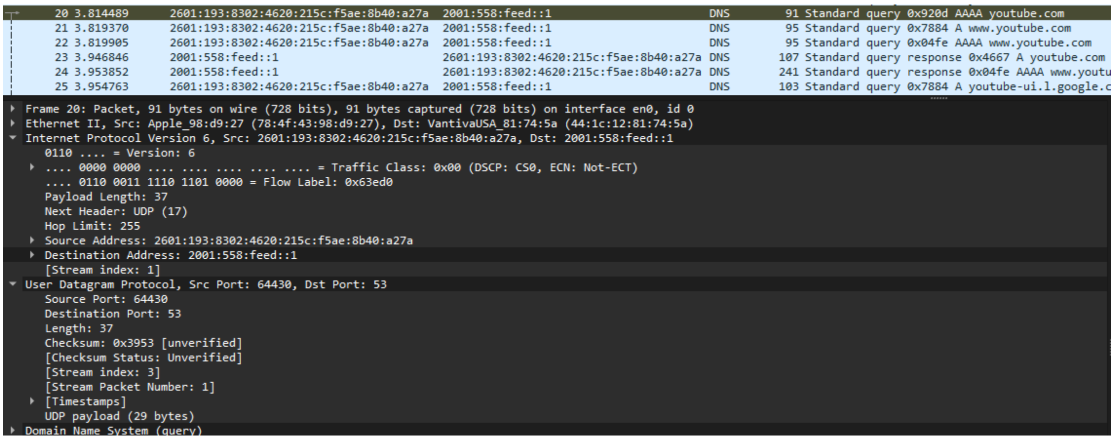
1. **Fragment Offset (分片偏移量)**：從 0 變成了 1480。
2. **Header Checksum (標頭檢查碼)**：因為 Offset 數值改變了，整個 IP 標頭的內容也隨之改變，因此檢查碼必須重新計算，兩者的數值會不一樣。

19. Now find the IP datagram containing the third fragment of the original UDP segment. What information in the IP header indicates that this is the last

fragment of that segment?

判斷是最後一個分片的關鍵在於，該封包 IP 標頭中的 **More fragments 旗標會變回 0 (Not set)**，這代表它後面已經沒有其他碎塊了。

Part 3: IPv6



20. What is the IPv6 address of the computer making the DNS AAAA request? This is the source address of the 20th packet in the trace. Give the IPv6 source address for this datagram in the exact same form as displayed in the Wireshark window .[5]

Source Address: 2601:193:8302:4620:215c:f5ae:8b40:a27a

21. What is the IPv6 destination address for this datagram? Give this IPv6 address in the exact same form as displayed in the Wireshark window.

Destination Address: 2001:558:feed::1

22. What is the value of the flow label for this datagram?

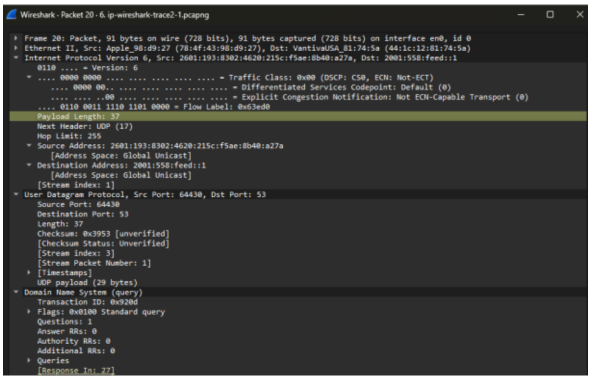
.... 0110 0011 1110 1101 0000 = Flow Label: 0x63ed0

23. How much payload data is carried in this datagram?

Payload Length: 37

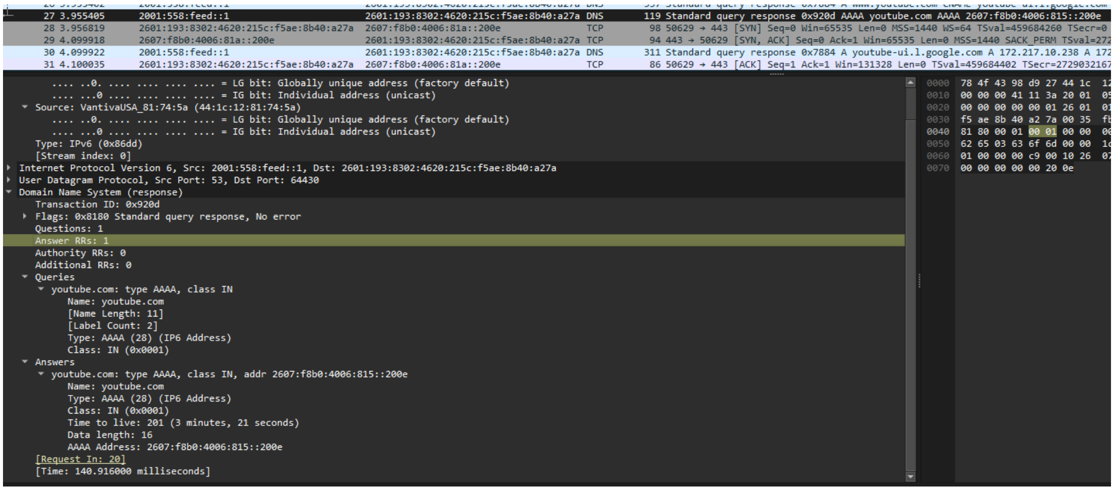
24. What is the upper layer protocol to which this datagram’s payload will be delivered at the destination?

Next Header: UDP (17)



25. How many IPv6 addresses are returned in the response to this AAAA request?

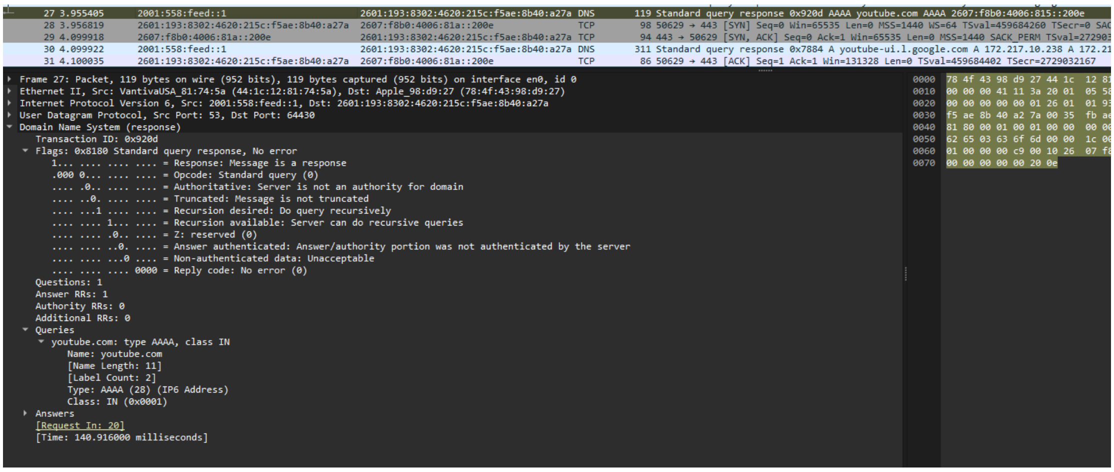
Answer RRs: 1



26. What is the first of the IPv6 addresses returned by the DNS for youtube.com (in the *ip-wireshark-trace2-1.pcapng* trace file, this is also the address that is numerically the smallest)? Give this IPv6 address in the exact same shorthand form as displayed in the Wireshark window.

2607:f8b0:4006:815::200e

27 3.955405 2001:558:feed::1 2601:193:8302:4620:215c:f5ae:8b40:a27a DNS 119 Standard query response 0x920d AAAA youtube.com AAAA 2607:f8b0:4006:815::200e


27 3.955405 2001:558:feed::1 2601:193:8302:4620:215c:f5ae:8b40:a27a DNS 119 Standard query response 0x920d AAAA youtube.com AAAA 2607:f8b0:4006:815::200e
28 3.956819 2601:193:8302:4620:215c:f5ae:8b40:a27a 2607:f8b0:4006:81a::200e TCP 98 50629 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1440 WS=64 TSval=459684260 TSecr=0 SAC
29 4.099918 2607:f8b0:4006:81a::200e 2601:193:8302:4620:215c:f5ae:8b40:a27a TCP 94 443 → 50629 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1440 SACK_PERM TSval=272903
30 4.099922 2001:558:feed::1 2601:193:8302:4620:215c:f5ae:8b40:a27a DNS 311 Standard query response 0x7884 A youtube-ui.l.google.com A 172.217.10.238 A 172.21
31 4.100035 2601:193:8302:4620:215c:f5ae:8b40:a27a 2607:f8b0:4006:81a::200e TCP 86 50629 → 443 [ACK] Seq=1 Ack=1 Win=131328 Len=0 TSval=459684402 TSecr=2729032167
Frame 27: Packet, 119 bytes on wire (952 bits), 119 bytes captured (952 bits) on interface en0, id 0
Ethernet II, Src: VantivaUSA_81:74:5a (44:1c:12:81:74:5a), Dst: Apple_98:d9:27 (78:4f:43:98:d9:27)
Internet Protocol Version 6, Src: 2001:558:feed::1, Dst: 2601:193:8302:4620:215c:f5ae:8b40:a27a
User Datagram Protocol, Src Port: 53, Dst Port: 64430
Domain Name System (response)
Transaction ID: 0x920d
Flags: 0x8180 Standard query response, No error
1... .... = Response: Message is a response
.000 0... .... = Opcode: Standard query (0)
.... .0.. .... = Authoritative: Server is not an authority for domain
.... ..0. .... = Truncated: Message is not truncated
....1 .... = Recursion desired: Do query recursively
.... 1... .... = Recursion available: Server can do recursive queries
.... .0.. .... = Z: reserved (0)
....0. .... = Answer authenticated: Answer/authority portion was not authenticated by the server
....0 .... = Non-authenticated data: Unacceptable
.... 0000 = Reply code: No error (0)
Questions: 1
Answer RRs: 1
Authority RRs: 0
Additional RRs: 0
Queries
youtube.com: type AAAA, class IN
Name: youtube.com
[Name Length: 11]
[Label Count: 2]
Type: AAAA (28) (IP6 Address)
Class: IN (0x0001)
Answers
[Request In: 20]
[Time: 140.916000 milliseconds]
0000 78 4f 43 98 d9 27 44 1c 12 81
0010 00 00 00 41 11 3a 20 01 05 58
0020 00 00 00 00 00 01 26 01 01 93
0030 f5 ae 8b 40 a2 7a 00 35 fb ae
0040 81 80 00 01 00 01 00 00 00 00
0050 62 65 03 63 6f 6d 00 00 1c 00
0060 01 00 00 00 c9 00 10 26 07 f8
0070 00 00 00 00 00 20 0e